

ARTIFICIAL INTELLIGENCE
FAKE NEWS DETECTION USING NATURAL
LANGUAGE PROCESSING (NLP) :

NAME : DINESHKUMAR S

NM ID : au513521104012

COLLEGE: AMCET

PHASE 5 – FINAL SUBMISSION :

PROBLEM DESCRIPTION :

The problem is to develop a fake news detection model using a Kaggle dataset. The goal is to distinguish between genuine and fake news articles based on their titles and text. This project involves using **Natural Language Processing (NLP)** techniques to preprocess the text data, building a **machine learning model** for classification, and evaluating the model's performance.

Now-a-days with the ease of content creation and due to the wide spread access of internet all over the world , fake news has become a major problem . This type of fake news may be misleading and may give the wrong information about a event which is dangerous.

The spread of fake news erodes trust in media and information sources, making it increasingly difficult for individuals to make informed decisions based on accurate information.

Fake news can influence public opinion, elections, and social dynamics, leading to harmful consequences for individuals and society as a whole .

So this project will be focused on detecting the fake news in the internet with the help of the NLP and by building a machine learning model using **Python** .

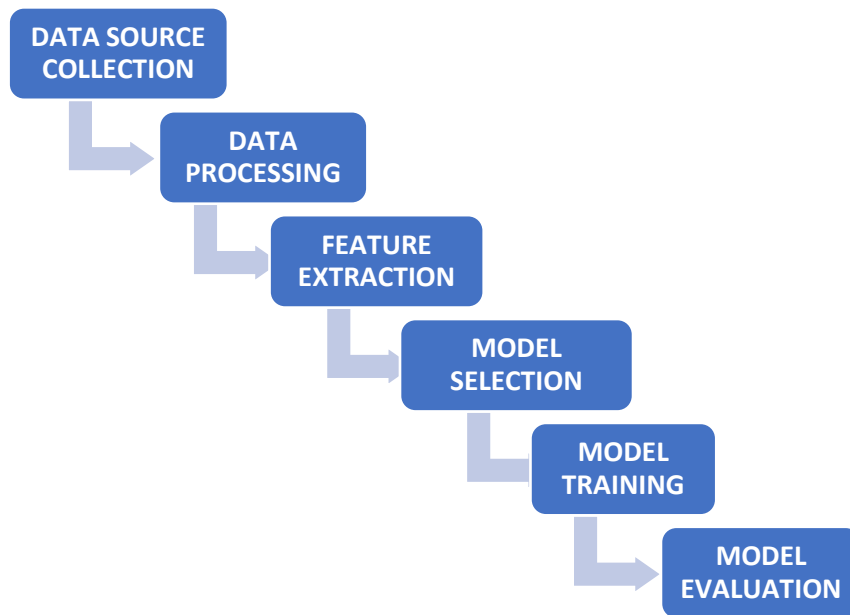
DESIGN THINKING :

The design thinking for this project includes **six steps** , they are

1. Data source collection
2. Data Processing

3. Feature extraction
4. Model Selection
5. Model Training
6. Model Evaluation

Let us see about the each phase in detail.



1. DATA SOURCE COLLECTION :

In this step **the required dataset** for the fake news detection is collected .For this process the classical dataset for fake detection from the Kaggle is taken . This dataset consists of the true news and the fake news and so by using this dataset it will be effective for the training of the ML model.

The link for the dataset we will be using for this project is :

<https://www.kaggle.com/clmentbisaillon/fake-and-real-news-dataset>

2. DATA PRE-PROCESSING :

Before feeding text data into **the LSTM** model, preprocess it by performing the following steps:

Tokenization: Tokenization is the process of **splitting text into individual words** or subwords. This step breaks down the text into its constituent elements, making it more manageable for further processing. For example, the sentence "I love deep learning" would be tokenized into ["I", "love", "deep", "learning"].

Lowercasing: Convert all **text to lowercase**. This ensures that words with different capitalizations are treated as the same word. For instance, "Apple" and "apple" become "apple."

Removing Stop Words: Remove **common words** like "and," "the," "is," etc., that may not carry much meaning.

Removing Special Characters: Eliminate **punctuation** and non-alphanumeric characters.

Proper data preprocessing ensures that the LSTM model receives **clean, structured, and meaningful input data**, which is essential for effective learning and improved model performance in fake news detection and other NLP tasks.

3. FEATURE EXTRACTION :

In this step the words are converted into the numerical features. This can be done by utilizing the techniques like **TF-IDF** (Term Frequency-Inverse Document Frequency) or **word embeddings** to convert text into numerical features.

4. MODEL SELECTION :

The core of fake news detection using NLP is **the LSTM model**. Design an innovative LSTM architecture that incorporates the following layers:

Embedding Layer:

Convert **numerical vectors (word embeddings) into continuous representations**.

LSTM Layers:

Stack multiple LSTM layers to capture sequential dependencies effectively.

Dropout Layers:

Prevent overfitting by randomly dropping out a fraction of neurons during training.

Dense Layers:

Add one or more dense layers for classification.

Output Layer:

Use a **sigmoid activation function** to produce a binary classification result (fake or real).

The LSTM model is used in this fake news detection model because of its simplicity and high accuracy . It consumes less training effort than BERT and so I chose to use LSTM model in this project .

5. MODEL TRAINING:

In this step the **preprocessed and cleaned data** is used to train the model with the help of the classification algorithms . The training process involves **finding the underlying patterns** in the dataset and enabling the model to perform **accurate predictions** ,etc .

The model should be trained with the **various forms of the preprocessed data** to improve its accuracy . The more data we feed to the model the **more accurate results** will be generated by the model .

6. MODEL EVALUATION :

In this step the trained model is evaluated with **various inputs** . The model is evaluated based on the metrics like **accuracy, precision, recall, F1-score, and ROC-AUC**.

DEVELOPMENT STEPS :

In this development process we are going to import required modules , load the dataset and preprocess the data for the training process .

1. IMPORTING LIBRARIES :

First we need to import the required modules need to perform the loading and preprocessing of the data .

Before importing the modules we need to install the modules .

```
pip install --upgrade tensorflow
pip install plotly
pip install --upgrade nbformat
pip install nltk
pip install spacy
pip install WordCloud
pip install gensim
pip install jupyterthemes
```

The tensorflow module is used for the machine learning process . The nltk (Natural Language Toolkit) is a library for natural language processing .Spacy is a open source software library for advanced natura language processing .Gensim is an open source library for unsupervised topic modelling and natural language processing .

After installing the requied packages now we need to import the modules for our project .

```
import nltk
nltk.download('punkt')

import tensorflow as tf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud, STOPWORDS
import nltk
import re
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
import gensim
from gensim.utils import simple_preprocess
from gensim.parsing.preprocessing import STOPWORDS
# import keras
from tensorflow.keras.preprocessing.text import one_hot, Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense, Flatten, Embedding, Input, LSTM, Conv1D, MaxPool1D, Bidirectional
from tensorflow.keras.models import Model
```

These modules will be used throughout the development process from the data loading and preprocessing to training the model and model evaluation .

2. **LOADING THE DATASETS :**

Now we need to load the required dataset in our program .

We use the fake news dataset form the Kaggle for this project . The link for the dataset is

<https://www.kaggle.com/clmentbisaillon/fake-and-real-news-dataset> .

The dataset is downloaded and the two files “True.csv” and “Fake.csv” are moved in the folder named “data” .

Now the pandas library is used to load the dataset into our program

```
df_true = pd.read_csv("data/True.csv")
df_fake = pd.read_csv("data/Fake.csv")
```

Now the dataset is loaded into the program .Now we can check the data is loaded correctly or not with the following code .

```
df_true.head()
```

It displays the first 5 items of the dataset . Similarly we can do for the fake data .

```
df_fake.head()
```

It displays the first 5 items of the fake news dataset .

Now we need to set the label for the fake and the real news datas .

For true values :

```
df_true['isfake'] = 1
```

For false values :

```
df_fake['isfake'] = 0
```

Sample output for `df_fake.head()`:

	title	text	subject	date
0	Donald Trump Sends Out Embarrassing New Year'...	Donald Trump just couldn t wish all Americans ...	News	December 31, 2017
1	Drunk Bragging Trump Staffer Started Russian ...	House Intelligence Committee Chairman Devin Nu...	News	December 31, 2017
2	Sheriff David Clarke Becomes An Internet Joke...	On Friday, it was revealed that former Milwauk...	News	December 30, 2017
3	Trump Is So Obsessed He Even Has Obama's Name...	On Christmas day, Donald Trump announced that ...	News	December 29, 2017
4	Pope Francis Just Called Out Donald Trump Dur...	Pope Francis used his annual Christmas Day mes...	News	December 25, 2017

3. DATA CLEANING :

The next step is to clean the dataset by removing unnecessary columns and datas that are not relevant for the training .

First we need to concatenate the real and fake datas together .

```
df = pd.concat([df_true, df_fake]).reset_index(drop = True)
```

This will concatenate the fake datas at the end of the real datas.

Now we are going to remove the date column which is not necessary for the model training .

```
df.drop(columns = ['date'], inplace = True)
```

This will remove the date column from the dataset .

Next step is to combine the “title” field and the “text” field and add them in the “original” field .

```
df['original'] = df['title'] + ' ' + df['text']
```

This will now create a new column “original” with the text and the title field combined .

Now after cleaning the datas we need to preprocess the data.

4. DATA PREPROCESSING :

Now we need to preprocess the dataset like removing the stopwords .

```
nltk.download("stopwords")
```

Next we need to obtain the additional stopwords from nltk .

```
from nltk.corpus import stopwords  
stop_words = stopwords.words('english')  
stop_words.extend(['from', 'subject', 're', 'edu', 'use'])
```

The next step is to remove the stop words and remove words with 2 or less characters using genism.

```
def preprocess(text):  
    result = []  
    for token in gensim.utils.simple_preprocess(text):  
        if token not in gensim.parsing.preprocessing.STOPWORDS and  
len(token) > 3 and token not in stop_words:  
            result.append(token)  
  
    return result
```

Now we apply this function to the dataframe .

```
df['clean'] = df['original'].apply(preprocess)
```

The original data is

```
df['original'][0]
```

Output:

'As U.S. budget fight looms, Republicans flip their fiscal script....'

Now the cleaned up data after removing the stopwords is

```
print(df['clean'][0])
```

Output:

['budget', 'fight', 'looms', 'republicans', 'flip', 'fiscal', 'script',]....

Next we need to obtain the total words present in the dataset .

```
list_of_words = []  
for i in df.clean:  
    for j in i:
```



```
list_of_words.append(j)
```

Find the length of the list of words

```
len(list_of_words)
```

Output : 9276947

Then find the total number of unique words

```
total_words = len(list(set(list_of_words)))  
total_words
```

Output : 108704

```
df['clean_joined'] = df['clean'].apply(lambda x: " ".join(x))
```

The output of the df:

	title	text	subject	isfake	original	clean	clean_joined
0	As U.S. budget fight looms, Republicans flip t...	WASHINGTON (Reuters) - The head of a conservat...	politicsNews	1	As U.S. budget fight looms, Republicans flip t...	[budget, fight, looms, republicans, flip, fisc...	budget fight looms republicans flip fiscal scr...
1	U.S. military to accept transgender recruits o...	WASHINGTON (Reuters) - Transgender people will...	politicsNews	1	U.S. military to accept transgender recruits o...	[military, accept, transgender, recruits, mond...	military accept transgender recruits monday pe...
2	Senior U.S. Republican senator: 'Let Mr. Muell...	WASHINGTON (Reuters) - The special counsel inv...	politicsNews	1	Senior U.S. Republican senator: 'Let Mr. Muell...	[senior, republican, senator, mueller, washing...	senior republican senator mueller washington r...
3	FBI Russia probe helped by Australian diplomat...	WASHINGTON (Reuters) - Trump campaign adviser ...	politicsNews	1	FBI Russia probe helped by Australian diplomat...	[russia, probe, helped, australian, diplomat, ...	russia probe helped australian diplomat washin...
4	Trump wants Postal Service to charge 'much mor...	SEATTLE/WASHINGTON (Reuters) - President Donal...	politicsNews	1	Trump wants Postal Service to charge 'much mor...	[trump, wants, postal, service, charge, amazon...	trump wants postal service charge amazon shipm...
...	...	...	...	...	...	...	...
44893	McPain: John McCain Furious That Iran Treated	21st Century Wire says As 21WIRE reported earl...	Middle-east	0	McPain: John McCain Furious That Iran Treated	[mcpain, john, mccain, furious, iran, treated,...	mcpain john mccain furious iran treated sailor...
44894	JUSTICE? Yahoo Settles E- mail Privacy Class-ac...	21st Century Wire says It s a familiar theme. ...	Middle-east	0	JUSTICE? Yahoo Settles E- mail Privacy Class-ac...	[justice, yahoo, settles, mail, privacy, class...	justice yahoo settles mail privacy class actio...

5. TOKENIZATION :

Now let us perform the tokenization

```
from sklearn.model_selection import train_test_split  
x_train, x_test, y_train, y_test = train_test_split(df.clean_joined,  
df.isfake, test_size = 0.2)  
from nltk import word_tokenize
```

```
tokenizer = Tokenizer(num_words = total_words)
tokenizer.fit_on_texts(x_train)
train_sequences = tokenizer.texts_to_sequences(x_train)
test_sequences = tokenizer.texts_to_sequences(x_test)
;
```

Now we can print the train_sequences:

```
print("The encoding for document\n",df.clean_joined[0],"\n is : ",train_sequences[0])
```

Output :

The encoding for document
 budget fight looms republicans flip fiscal script washington.....
 [2817, 8135, 2981, 4196, 666, 1, 6236,541 ,27, 2444]

```
padded_train = pad_sequences(train_sequences,maxlen = 40, padding = 'post', truncating = 'post')
padded_test = pad_sequences(test_sequences,maxlen = 40, truncating = 'post')
```

```
for i,doc in enumerate(padded_train[:2]):
    print("The padded encoding for document",i+1," is : ",doc)
```

Output :

The padded encoding for document 1 is : [2817 8135 2981 4196 666
 1 6236 541 27 2444 1713 205
 2817 8135 5753 666 6236 10 1 45 541 1 2160 43225
 3477 183 1311 270 8135 641 1256 2444 17467 183 623 3126
 1240 19 1240 3836]
 The padded encoding for document 2 is : [133 2174 1732 2842 370
 120 442 226 9784 16805 17154 8
 133 8685 1732 322 11 370 1640 5264 4644 1808 442 226
 13539 942 1375 12180 390 90 1956 211 357 3549 2 11
 705 975 1956 743]

Thus the data is now converted into numbers using the tokenizer the next step is to train the model and evaluate it that will be done in the next phase project .

DEVELOPMENT STEPS FOR TRAINING THE MODEL :

1. BUILDING THE MODEL :

In this step we are going to build our model for the fake news detection. We use a sequential model for training .

```
# Sequential Model
model = Sequential()

# embedding layer
model.add(Embedding(total_words, output_dim = 128))
# model.add(Embedding(total_words, output_dim = 240))

# Bi-Directional RNN and LSTM
model.add(Bidirectional(LSTM(128))) # no of neurons

# Dense layers
model.add(Dense(128, activation = 'relu'))
model.add(Dense(1, activation= 'sigmoid')) # reason: we do binary
classification here
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['acc'])
model.summary()
```

We use the LSTM(Long Short Term Memory) for training the model and we specify the number of neurons we need for training the model .

Then we add the activation functions like relu (Rectified Linear Unit) and sigmoid to the model . Then get the summary of the model .

OUTPUT:

Model: "sequential_1"

Layer (type)	Output Shape	Param #
--------------	--------------	---------

embedding_1 (Embedding)	(None, None, 128)	13914112
-------------------------	-------------------	----------

bidirectional_1 (Bidirection)	(None, 256)	263168
-------------------------------	-------------	--------

dense_2 (Dense)	(None, 128)	32896
-----------------	-------------	-------

dense_3 (Dense)	(None, 1)	129
-----------------	-----------	-----

Total params: 14,210,305

Trainable params: 14,210,305

Non-trainable params: 0

This is the output generated for the model building .

2. **MODEL TRAINING :**

The next step after building the model is to train the model.

Here we use 2 epochs for training the model .

```
y_train = np.asarray(y_train)
```

We convert the data for training into an array using the above code .

```
model.fit(padded_train, y_train, batch_size = 64, validation_split = 0.1, epochs = 2)
```

This is the code for training the model . We use the padded_train that we generated during the tokenization phase in the AI_PHASE3 and the y_train to train the model .

We mention some essential parameters for training such as batch_size=64 ,validation_split=0.1 and epochs=2 .

OUTPUT:

Epoch 1/2

506/506 [=====] - 150s 297ms/step -
loss: 0.0422 - acc: 0.9829 - val_loss: 0.0090 - val_acc: 0.9983

Epoch 2/2

506/506 [=====] - 126s 248ms/step -
loss: 0.0014 - acc: 0.9997 - val_loss: 0.0114 - val_acc: 0.9978

3. ASSESS THE MODEL PERFORMANCE :

The next step after training the model is to assess the model performance .

First we make the prediction using the model.

```
pred = model.predict(padded_test)
```

If the predicted value is greater than 0.5 then it is real or else it is false .To implement this we write the below coding .

```
prediction = []  
for i in range(len(pred)):  
    if pred[i].item() > 0.5:  
        prediction.append(1)  
    else:  
        prediction.append(0)
```

Now we measure the accuracy of the model using the sklearn.metrics library .

```
from sklearn.metrics import accuracy_score  
  
accuracy = accuracy_score(list(y_test), prediction)  
  
print("Model Accuracy : ", accuracy)
```

OUTPUT:

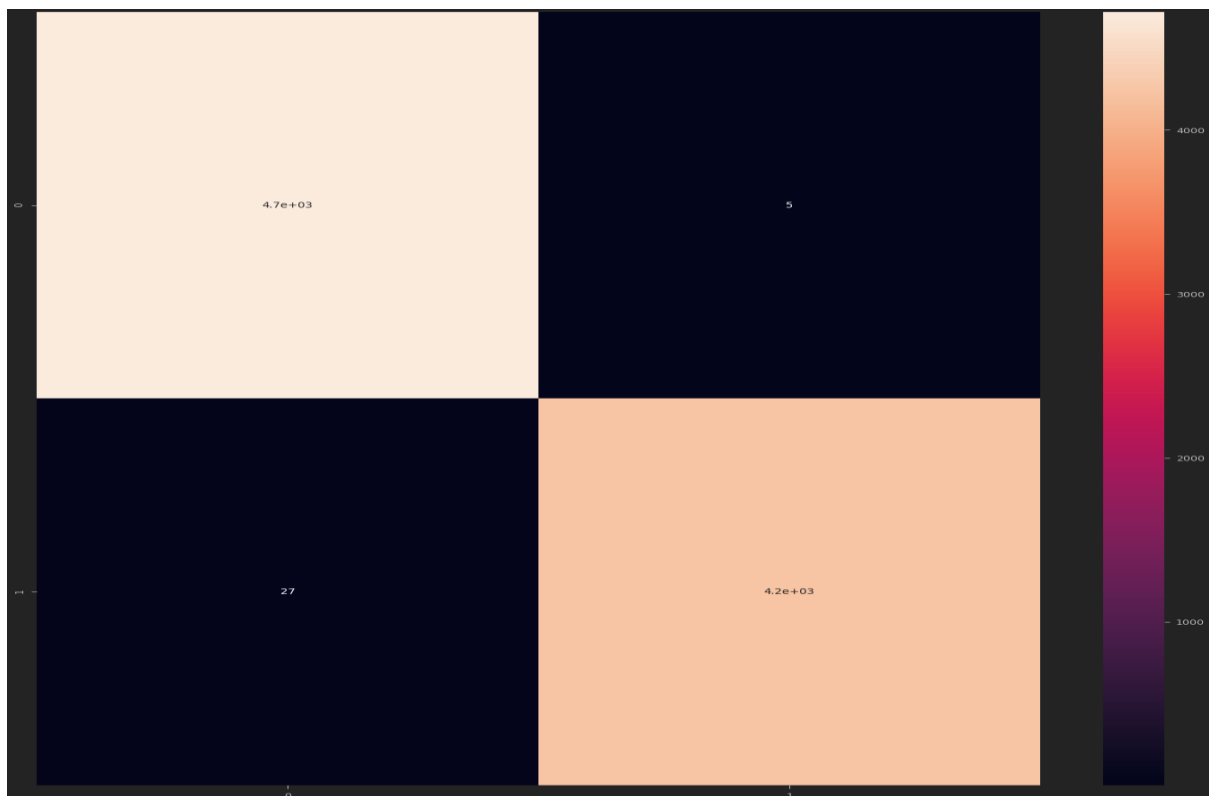
Model Accuracy : 0.9869710467706013

Thus the accuracy of our trained model is nearly 98% . If the accuracy of our model is low then we can retrain the model .

Final step is to visualize the model using the confusion matrix . We use the sklearn.metrics for this .

```
from sklearn.metrics import confusion_matrix  
cm = confusion_matrix(list(y_test), prediction)  
plt.figure(figsize = (25, 25))  
sns.heatmap(cm, annot = True)
```

OUTPUT :



This is the confusion matrix for our model . Thus the model training and measuring the accuracy of the model is finally executed successfully.

CONCLUSION:

These are the steps that were followed to develop the Fake News Detection model.

The source code for the model is uploaded in the github and the link is given below :

https://github.com/dineshkumar-2003/FAKE_NEWS_DETECTION_USING_NLP/blob/850a43a7be91ab3ebc34776cdf777dfed8fc6375/Fake%20News%20Detection%20.ipynb

The input and output are compiled and explained in detail in this jupyter notebook file .

Thus the problem statement ,data preprocessing , model training and the model evaluation for the Fake News Detection model is successfully completed.